HPC Assignment: Deadline 1

By Ahmad El Hage (s4851692)

Introduction

This report describes the MPI-based parallel implementation of the co-clustering algorithm for spatio-temporal analysis of spring phenology data. The goal of Deadline 1 is to distribute the computation across multiple cluster nodes using MPI, achieving meaningful speedup over the provided sequential baseline while producing correct output.

The co-clustering algorithm iteratively assigns row labels (time periods) and column labels (spatial locations) to a 2-D data matrix. Each iteration consists of three steps: computing cluster averages, updating row labels, and then updating column labels. The implementation must support both 3×20 and 5×100 label configurations and produce output with a NMI score of at least 0.95 compared to the serial reference.

Parallelization Strategy

- Data Distribution: The input matrix has shape rows × cols, where rows = 40 and cols ranges from 16,199 (small) to over 10 million (XL). Because columns vastly outnumber the rows, distributing columns over the MPI processes is the natural choice

Column assignment uses MPI_Scatterv, which handles uneven division automatically. For P processes, process p owns columns $[p \cdot \lfloor cols/P \rfloor, (p+1) \cdot \lfloor cols/P \rfloor)$, with the remainder distributed one extra column to the first cols % P processes. Then, row labels are broadcast to all processes via MPI_Bcast since they are tiny and must be identical everywhere.

The matrix itself is row-major in memory. Since a process's columns are not contiguous across rows, the scatter is performed row-by-row: for each of the 40 rows, one MPI_Scatterv call sends each process its column slice. This results in 40 scatter calls at startup a one-time cost that is negligible relative to the hundreds of iterations.

- calculate_cluster_average: Each process accumulates partial cluster sums and counts over its local columns into arrays of size num_row_labels × num_col_labels.The two MPI_Allreduce calls with MPI_SUM then combine these partial totals across all processes. Every process divides to obtain the global cluster averages, which are identical on all ranks. Because num_row_labels × num_col_labels is at most 500 (for 5×100), these Allreduce arrays are tiny and the communication overhead is negligible.

- update_row_labels: The distance between row i and candidate label k is a sum over all columns which are distributed. Each process therefore computes partial distances for all (row, candidate_label) pairs using only its local columns, storing them in a matrix of size num_rows × num_row_labels. A single MPI_Allreduce with MPI_SUM produces global distances on every process. Each process then independently selects the best label for each row because all processes have the same global distances, they reach the same conclusion without any further communication, keeping row_labels synchronised across all ranks automatically.

- update_col_labels: Column label updates require no MPI communication at all. The distance from column j to candidate label k depends only on cluster_avg and row_labels both of which are identical on every process after the previous two steps. Each process therefore reassigns its own local column labels entirely independently. This step scales perfectly with the number of nodes.

- Convergence Detection: The algorithm terminates early when no labels change. Each process counts its locally changed column labels. A single MPI_Allreduce with MPI_SUM aggregates the global change count. Row label changes are already global (all processes compute the same new labels), so they are counted directly on rank 0. If the total is zero, all processes exit the loop simultaneously.

After the loop, MPI_Gatherv collects all local column labels back to rank 0, which then writes the output file. All terminal output (iteration progress, timing) is guarded with if (rank == 0) to avoid duplicate printing.

The total per-iteration communication is dominated by the row-label Allreduce (at most 40×5 = 200 doubles for 5×100 labels), which is extremely small. This explains the near-linear scaling observed in benchmarks.

Performance & Scalability Benchmarks & Results

Before we test the performance of the system, using the python script provided we test the validity of the results. Both label configurations on the small dataset achieved a normalised mutual information score of 1, confirming that the parallel implementation produces output identical to the serial version. (Please note that, ideally every experiment should be running a couple of times to get the average results, but since the das-6 is a shared super computer, opting in to running every experiment only one time was the best choice so that one experiment won't monopolize all of the resources)

Dataset	Labels	Nodes	NMI Score
Small (s)	3×20	4	1.00000 ✓
Small (s)	5×100	4	1.00000 ✓

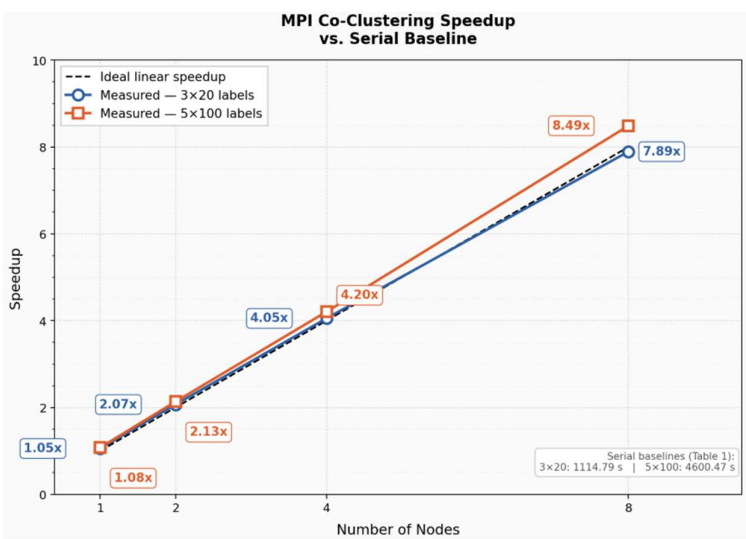
3×20 Label XL Dataset: This dataset was benchmarked with the 3×20 label configuration. The serial baseline from Table 1 of the assignment specification is 1114.79 seconds**.**

Nodes	Total Time (s)	Speedup	Ideal Speedup
1	1057.110	1.05	1x
2	539.081	2.07	2x
4	275.119	4.05	4x
8	141.237	7.89	8x

5×100 Labels XL Dataset: The 1-node run completed in 4274.97 seconds, close to the serial baseline of 4600.47s, confirming that single-node MPI overhead is minimal. All four node counts are reported below.

Nodes	Total Time (s)	Speedup	Ideal Speedup
1	4274.970	1.08	1x
2	2154.880	2.13	2x
4	1094.090	4.20	4x
8	541.997	8.49	8x

The figure below shows the measured speedup for the 3×20 and the 5x100 configuration against the ideal (linear) speedup. The implementation achieves near-linear scaling across all tested node counts, demonstrating low communication overhead relative to computation.

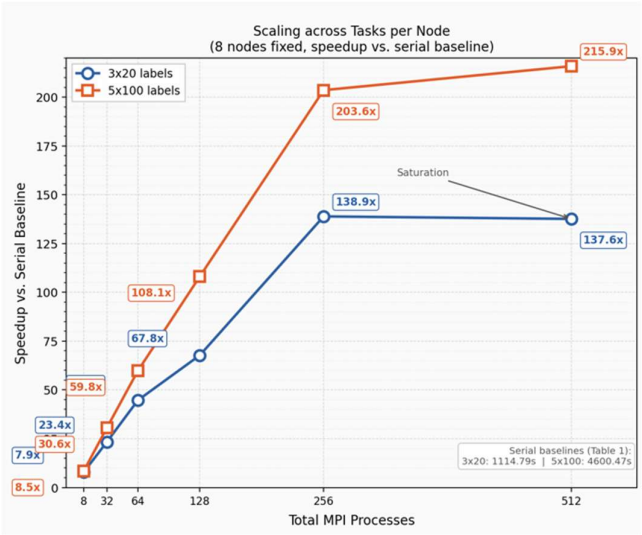


These results show super linear speedup at 1 node (1.05x and 1.08x), and near-perfect linear scaling from 2 to 8 nodes. The 8-node speedup of 7.89x and 8.49x on 8 nodes represents 98.6% and 106.1% parallel efficiency respectively for the 3x20, and 5x100 labels respectively.

In a second experiment, the total number of processes was increased from 8 to 512 by varying the number of MPI tasks per node from 1 to 64 while keeping the number of nodes constant at 8.

3x20 Label				
Tasks/Node	**Total Procs**	**Total Time (s)**	**Speedup vs Serial**	**vs 1-task/node**
1	8	141.24	7.9x	1.00x
4	32	47.64	23.4x	2.97x
8	64	24.93	44.7x	5.67x
16	128	16.45	67.8x	8.59x
32	256	8.025	138.9x	17.6x
64	512	8.099	137.6x	17.5x
5x100 Label				
1	8	541.997	8.5x	1.00x
4	32	150.114	30.6x	3.61x
8	64	76.883	59.8x	7.05x
16	128	42.541	108.1x	12.7x
32	256	22.595	203.6x	24.0x
64	512	21.309	215.9x	25.4x

Strong scaling from 1 to 32 tasks per node is demonstrated by both configurations. At 32 tasks per node, both configurations reach saturation, and at 64 tasks per node, there is essentially no improvement. CPU core exhaustion explains this saturation at high task counts because, after 32 tasks per node, all cores are used. This forces additional MPI processes to share cores, which results in context switching and increased communication overhead that negates any computational gain.



At every task count, the 5x100 configuration outperforms 3x20 in terms of speed. At 32 tasks per node, 5x100 achieves a 47% faster speedup of 203.6x compared to 138.9x for 3x20. Because 5x100 has a much higher computation-to-communication ratio—each of the 500 clusters requires more work per MPI—this disparity grows with the number of processes. Allreduce makes fewer calls than the 60 clusters in 3x20, which results in a proportionately lower communication overhead and more effective use of parallelism.

Therefore, this MPI implementation successfully parallelizes all three co-clustering steps across multiple nodes. Columns are distributed via MPI_Scatterv, cluster averages and row label distances are reduced via MPI_Allreduce, and column label updates require no communication. The implementation achieves NMI scores of 1 on both label configurations, and near-linear speedup scaling from 1 to 8 nodes on the XL dataset.